

Gioush 大队 2026 夏季常规赛

夏季训练赛

GSOI 2026

第十三试

时间：2026 年 8 月 16 日 14:00 ~ 17:30

题目名称	天地皆可往	单舟见京杭	新水濯旧隍	云止聆佳响
题目类型	传统型	交互型	交互型	传统型
目录	roam	canal	moat	echo
可执行文件名	roam	canal	moat	echo
输入文件名	roam.in	无	无	echo.in
输出文件名	roam.out	无	无	echo.out
每个测试点时限	1.0 秒	8.0 秒	8.0 秒	2.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	20	4	4	20
测试点是否等分	是	否	否	是

提交源程序文件名

对于 C++ 语言	roam.cpp	canal.cpp	moat.cpp	echo.cpp
-----------	----------	-----------	----------	----------

编译选项

对于 C++ 语言	-O2 -std=c++14
-----------	----------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 main() 的返回值类型必须是 int，程序正常结束时的返回值必须是 0。
3. 因违反以上两点而出现的错误或问题，申诉时一律不予受理。
4. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
5. 选手提交的程序源文件必须不大于 100KB。
6. 程序可使用的栈空间内存限制与题目的内存限制一致。

天地皆可往 (roam)

【题目背景】

龙震于疆，万里宁壤，天地皆可往。

【题目描述】

大陆上共有 n 片等待安定的疆土。游龙希望踏遍这些疆土，使边境重归安宁。

游龙拥有一种名为**龙威**的力量。在前往第 i 片疆土前，游龙当前的**龙威**必须不少于 a_i 。完成当地的事务后，游龙的**龙威**会变化 b_i ，其中 b_i 可以为负数。保证 $a_i + b_i \geq 0$ ，因此，只要游龙能够进入一片疆土，离开时的**龙威**就不会小于 0。

游龙可以按照任意顺序前往这些疆土，每片疆土必须且只能前往一次。在整个过程中，游龙需要在前往每片疆土前满足对应的**龙威**要求。

请你求出，为了使游龙最终能够踏遍所有疆土，游龙最少需要拥有多少初始**龙威**。

【输入格式】

从文件 `roam.in` 中读入数据。

本题包含多组测试数据。

输入的第一行包含两个整数 c, T ，分别表示测试点编号与测试数据组数。 $c = 0$ 表示该测试点为样例。

接下来依次输入每组测试数据。对于每组测试数据：

第一行包含一个整数 n ，表示疆土的数量。

接下来 n 行，第 i 行包含两个整数 a_i, b_i ，分别表示前往第 i 片疆土需要的最低**龙威**，以及完成当地事务后**龙威**的变化量。

【输出格式】

输出到文件 `roam.out` 中。

对于每组测试数据，输出一行一个整数，表示游龙踏遍所有疆土所需的最小初始**龙威**。

【样例 1 输入】

```
1 0 2
2 3
3 5 4
4 8 -3
5 4 2
```

```
6 4
7 6 -6
8 3 5
9 10 -2
10 8 -7
```

【样例 1 输出】

```
1 4
2 10
```

【说明/提示】**【样例 1 解释】**

对于第一组测试数据，游龙可以依次前往第 3, 1, 2 片疆土。若初始龙威为 4，每次完成当地事务后的龙威依次为 6, 10, 7，因此可以踏遍所有疆土。由于前往任意一片疆土都至少需要 4 点龙威，初始龙威不可能小于 4。

对于第二组测试数据，游龙可以依次前往第 2, 3, 4, 1 片疆土。若初始龙威为 10，每次完成当地事务后的龙威依次为 15, 13, 6, 0。可以证明，初始龙威小于 10 时无法踏遍所有疆土。

【样例 2】

见选手目录下的 *roam/roam2.in* 与 *roam/roam2.ans*。
该组样例符合测试点 1 ~ 2 的数据范围。

【样例 3】

见选手目录下的 *roam/roam3.in* 与 *roam/roam3.ans*。
该组样例符合测试点 3 ~ 5 的数据范围。

【样例 4】

见选手目录下的 *roam/roam4.in* 与 *roam/roam4.ans*。
该组样例符合测试点 6 ~ 10 的数据范围。

【样例 5】

见选手目录下的 *roam/roam5.in* 与 *roam/roam5.ans*。
该组样例符合测试点 11 ~ 15 的数据范围。

【样例 6】

见选手目录下的 *roam/roam6.in* 与 *roam/roam6.ans*。
该组样例符合测试点 16 ~ 20 的数据范围。

【数据范围】

对于 100% 的数据, 保证 $1 \leq T \leq 20, 1 \leq n \leq 2 \times 10^5, 1 \leq a_i \leq 10^9, -a_i \leq b_i \leq 10^9$ 。
对于同一个测试点, 保证 $\sum n \leq 2 \times 10^5$ 。

测试点编号	n	特殊性质
1 ~ 2	≤ 9	无
3 ~ 5	≤ 18	
6 ~ 10	$\leq 2 \times 10^5$	A
11 ~ 15		B
16 ~ 20		无

特殊性质 A: 对于所有 $1 \leq i \leq n$, 均有 $b_i \geq 0$ 。
特殊性质 B: 对于所有 $1 \leq i \leq n$, 均有 $b_i < 0$ 。

单舟见京杭 (canal)

本题为交互题。

【题目背景】

龙泽于汤，唤水筑乡，单舟见京杭。

【题目描述】

若干位旅客沿着运河一路向北前行。这条运河共有 n 段水道，每段水道的左右两侧各有一处景观。对于第 i 段水道，其左右两侧景观的**美观度**分别为 w_i 与 v_i 。

每位旅客经过一段水道时，都可以选择向左看或向右看。旅客最终收获的**幸福值**，等于沿途看到的所有不同景观的**美观度**之和。

运河蜿蜒曲折。在全部 $2n$ 处景观中，恰有两处实际上是同一处景观，只是旅客所处的位置发生了改变，才会从不同水道再次看到它。这两处景观位于不同的水道，且**美观度**相同。其余景观的**美观度**不要求互不相同。

若一位旅客同时选择看到这两处**重复景观**，那么这处景观的**美观度**只会对其**幸福值**产生一次贡献。

在旅程开始前，你已经知道所有 w_i, v_i ，却不知道哪两处景观是**重复景观**。你可以提前安排若干位旅客，分别决定每位旅客经过每段水道时看向哪一侧。每位旅客抵达终点后，都会向你报告获得的**幸福值**。

现在，你需要根据这些报告确定两处**重复景观**各自所在的水道与方向。每多安排一位旅客，都会打扰原本平静的行程，因此你希望使用的旅客数量尽可能少。

【实现细节】

选手不需要，也不应实现 `main` 函数。

选手需要确保提交的程序包含头文件 `canal.h`，即在程序开头加入以下代码：

```
1 #include "canal.h"
```

选手需要在提交的程序源文件 `canal.cpp` 中实现以下两个函数：

```
1 void Init(int c, int T);
```

- c, T 分别表示测试点编号与测试数据组数。 $c = 0$ 表示该测试点为样例。
- 对于每个测试点，该函数会在程序开始运行时被交互库调用恰好一次。

```
1 void Locate(int c, int n, std::vector<int> w,  
2             std::vector<int> v);
```

- c, n 分别表示测试点编号与水道数量。

- w 与 v 的长度均为 $n + 1$ 。对于 $1 \leq i \leq n$, $w[i]$ 与 $v[i]$ 分别表示第 i 段水道左侧与右侧景观的**美观度**。 $w[0]$ 与 $v[0]$ 没有实际意义。
 - 对于每组测试数据, 该函数会被交互库调用恰好一次。
- 在 `Locate` 函数中, 选手可以调用以下函数安排一位旅客:

```
1 long long Travel(std::vector<int> choice);
```

- `choice` 的长度必须为 $n + 1$ 。`choice[0]` 没有实际意义。
 - 对于 $1 \leq i \leq n$, `choice[i]` 必须为 0 或 1。0 表示旅客经过第 i 段水道时向左看, 1 表示向右看。
 - 该函数返回这位旅客最终获得的**幸福值**。
 - 选手需要确保交互库每次调用 `Locate` 时, 调用该函数的次数不超过 200。
- 选手需要调用以下函数给出答案:

```
1 void Answer(int x, int a, int y, int b);
```

- (x, a) 与 (y, b) 分别描述一处**重复景观**。其中 $1 \leq x, y \leq n$, $x \neq y$, $a, b \in \{0, 1\}$ 。
- $a = 0$ 表示第 x 段水道左侧的景观, $a = 1$ 表示右侧的景观。 b 的含义与之相同。
- 交换 (x, a) 与 (y, b) 的顺序仍视为同一答案。
- 选手需要确保交互库每次调用 `Locate` 时, 恰好调用该函数一次。
- 调用该函数后, 本组交互立即结束。此后不得继续调用 `Travel` 或 `Answer`, 且 `Locate` 函数应当结束运行。

交互库运行所需的时间与空间均计入本题的时间与空间限制。

【测试程序方式】

选手可以在本题目录下使用如下命令编译得到可执行文件:

```
1 g++ grader.cpp canal.cpp -o canal -O2 -std=c++14 -static
```

对于编译得到的可执行文件 `canal`:

- 可执行文件将从标准输入读入以下格式的数据:
 - 第一行包含两个非负整数 c, T , 分别表示测试点编号与测试数据组数。 $c = 0$ 表示该测试点为样例。
 - 接下来对于每组测试数据, 第一行包含一个正整数 n 。
 - 第二行包含 n 个正整数 $w_1, w_2, \dots, w_n$ 。
 - 第三行包含 n 个正整数 $v_1, v_2, \dots, v_n$ 。
 - 第四行包含四个整数 x, a, y, b , 表示两处**重复景观**。这些信息只会由本地测试交互库读取, 不会传入 `Locate` 函数。
- 若 T 次调用 `Answer` 给出的答案均正确, 可执行文件将输出以下格式的数据至标准输出:

- 输出的第一行为 **Correct!**。
 - 输出的第二行为 **Max travelers used: q** , 其中 q 表示所有测试数据中调用 **Travel** 次数的最大值。
 - 输出的第三行为 **Score ratio: $p\%$** , 其中 p 表示按照【评分方式】计算出的得分百分比。
- 若至少一次调用 **Answer** 给出的答案不正确, 则可执行文件只会向标准输出输出一行 **Wrong answer**。
 - 若调用 **Travel** 或 **Answer** 时传入的参数不符合要求, 调用次数超过上限, 或函数调用顺序不合法, 则可执行文件会向标准错误流输出错误信息并终止程序。

【样例 1 输入】

```
1 0 1
2 4
3 5 3 7 2
4 1 7 4 6
5 2 1 3 0
```

【样例 1 输出】

```
1 Correct!
2 Max travelers used: 5
3 Score ratio: 100%
```

【说明/提示】**【样例 1 解释】**

样例中, 第 2 段水道右侧与第 3 段水道左侧看到的是同一处景观, 这两处景观的美观度均为 7。

样例输出对应一种使用 5 位旅客并正确调用 **Answer(2,1,3,0)** 的策略。不同的合法策略可能产生不同的样例输出。

【样例 2】

见选手目录下的 *canal/canal2.in*。

该组样例符合测试点 1 的数据范围。

【样例 3】

见选手目录下的 *canal/canal3.in*。
该组样例符合测试点 2 的数据范围。

【样例 4】

见选手目录下的 *canal/canal4.in*。
该组样例符合测试点 3 的数据范围。

【样例 5】

见选手目录下的 *canal/canal5.in*。
该组样例符合测试点 4 的数据范围。

本题为交互题，附加样例只提供隐藏输入。测试程序的输出由选手程序采用的策略决定。

【数据范围】

对于 100% 的数据，保证 $1 \leq T \leq 20$ ， $2 \leq n \leq 10^5$ ， $1 \leq w_i, v_i \leq 10^9$ 。对于同一个测试点，保证 $\sum n \leq 2 \times 10^5$ 。

测试点编号	分值	n	特殊性质
1	15	≤ 7	无
2	20	$\leq 10^5$	A
3	40	≤ 100	无
4	25	$\leq 10^5$	无

特殊性质 A：两处**重复景观**都位于水道左侧，或都位于水道右侧。

【评分方式】

注意：

- 选手不应当通过非法方式获取交互库的内部信息，如试图直接读取两处**重复景观**的位置，或直接与标准输入、输出流进行交互。此类行为将被视为作弊。
- 交互库不是适应性的。每次调用 `Locate` 时，所有景观的**美观度**与两处**重复景观**的位置均已确定，不会随交互过程改变。
- 最终的评测交互库与本地测试交互库的实现不同。

若 `Answer` 给出的答案不正确，调用 `Travel` 或 `Answer` 时不符合以上要求，或调用次数超过上限，则相应测试点得 0 分。

在上述条件均满足的基础上，记一个测试点中调用 `Travel` 次数的最大值为 q ，该测试点的得分比例为

$$p(q) = \begin{cases} 1, & q \leq 80, \\ \frac{80}{q}, & 80 < q \leq 200. \end{cases}$$

若 $q \leq 80$ ，则该测试点取得全部分值。若 $80 < q \leq 200$ ，则该测试点取得对应分值的 $\frac{80}{q}$ 。

记表中该测试点的分值为 d ，则取整前的实际得分为 $dp(q)$ 。最终得分保留两位小数，并采用向下取整，也就是

$$\frac{\lfloor 100dp(q) \rfloor}{100}.$$

新水濯旧隍 (moat)

本题为交互题。

【题目背景】

龙华于旸，红旗漫卷，新水濯旧隍。

【题目描述】

前朝遗老的王宫里，有 n 个宫室，其中有一个宫室是**明光殿**，记为 1 号宫室，这 n 个宫室由 $n - 1$ 条宫内道路连通，也就是说，任意两个宫室 u, v 都能通过若干条宫内道路互相到达。第 i 个宫室中收藏着价值 w_i 的宝藏。每个 w_i 都是正整数。

旧王朝被推翻后，**清洗者**和**斥候**来到了**明光殿**，决定探测整个王宫。前朝留下的宫图记录了所有宫内道路，却没有记载任何一处藏宝量。因此，在行动开始前，王宫的结构是已知的，而每个 w_i 都是未知的。

以**明光殿**为根，宫室 u 的深度定义为从**明光殿**前往宫室 u 时经过的宫室数量。保证每个宫室至多与两个深度比它大 1 的宫室直接相连。令 h 表示所有宫室深度的最大值。

行动共进行 b 轮。每轮行动依次分为**探查阶段**与**清洗阶段**。

对于一个宫室 u ，将宫室 u 与所有从**明光殿**前往时必须经过宫室 u 的宫室共同组成的区域称为宫室 u 的**辖域**。在**探查阶段**，你可以让**斥候**调查任意一个宫室 u ，**斥候**会报告宫室 u 的**辖域**中当前剩余的藏宝量之和。你可以根据此前得到的所有报告决定下一次调查的位置，也可以随时结束本轮的**探查阶段**。

随后进入本轮的**清洗阶段**。你必须让**清洗者**选择一个此前没有清洗过的宫室 u 。**清洗者**会取得宫室 u 中的全部财宝，并将这个宫室的藏宝量清零。此后，本轮行动结束，下一轮调查所得的藏宝量之和也会随之改变。

而你作为领导者，需要根据**斥候**不断送回的消息，为**清洗者**依次确定 b 个互不相同的目标，使得最终获取的藏宝量尽可能多。视你获取的藏宝量多少，你将会获得不同的评价。

【实现细节】

选手不需要，也不应实现 `main` 函数。

选手需要确保提交的程序包含头文件 `moat.h`，即在程序开头加入以下代码：

```
1 #include "moat.h"
```

选手需要在提交的程序源文件 `moat.cpp` 中实现以下两个函数：

```
1 void Init(int c, int T);
```

- c, T 分别表示测试点编号与测试数据组数。 $c = 0$ 表示该测试点为样例。

- 对于每个测试点，该函数会在程序开始运行时被交互库调用恰好一次。

```
1 void Play(int c, int n, int b, std::vector<int> parent);
```

- c, n, b 分别表示测试点编号、宫室数量与行动轮数。
 - `parent` 的长度为 $n + 1$ 。其中 `parent[1]` 等于 0，对于 $2 \leq u \leq n$ ，`parent[u]` 表示从宫室 u 前往明光殿时经过的第一个宫室。
 - 选手可以由 `parent` 得知完整的王宫结构。每个宫室的藏宝量不会传入该函数。
 - 对于每组测试数据，该函数会被交互库调用恰好一次。
- 在 `Play` 函数中，选手可以调用下列函数。

```
1 long long Scout(int u);
```

- 该函数表示让斥候调查宫室 u ，其中 $1 \leq u \leq n$ 。
- 该函数返回宫室 u 的辖域中当前剩余的藏宝量之和。
- 该函数只能在一轮行动的探查阶段调用。
- 设王宫的高度为 h 。选手需要确保交互库每次调用 `Play` 时，调用该函数的次数不超过 $2bh + 20$ 。

```
1 long long Clean(int u);
```

- 该函数结束当前一轮的探查阶段，并让清洗者在清洗阶段清洗宫室 u ，其中 $1 \leq u \leq n$ 。
- 宫室 u 不能在此前的清洗阶段中被选择过。
- 该函数返回本次取得的藏宝量。调用结束后，宫室 u 的藏宝量变为 0，并立即开始下一轮的探查阶段。
- 每组测试数据中，该函数必须恰好调用 b 次。第 b 次调用结束后，`Play` 函数应当结束运行。

交互库运行所需的时间与空间均计入本题的时间与空间限制。

【测试程序方式】

选手可以在本题目录下使用如下命令编译得到可执行文件：

```
1 g++ grader.cpp moat.cpp -o moat -O2 -std=c++14 -static
```

对于编译得到的可执行文件 `moat`：

- 可执行文件将从标准输入读入以下格式的数据：
 - 第一行包含两个非负整数 c, T ，分别表示测试点编号与测试数据的组数。
 $c = 0$ 表示该测试点为样例。
 - 接下来对于每组测试数据，第一行包含两个正整数 n, b 。
 - 第二行包含 n 个正整数 $w_1, w_2, \dots, w_n$ 。

- 接下来 $n - 1$ 行，每行包含两个正整数 u, v ，表示宫室 u, v 之间有一条宫内道路。
- 若所有交互均合法，可执行文件将输出以下格式的数据至标准输出：
 - 输出的第一行为 **Finished!**。
 - 接下来 T 行中，第 i 行形如 **Case i: treasure = s, score ratio = p%**，其中 s 表示第 i 组测试数据中取得的藏宝量， p 表示该组测试数据的得分百分比。
 - 输出的最后一行为 **Score ratio: p%**，其中 p 表示整个测试点的得分百分比。
- 若发生非法调用，可执行文件会向标准错误流输出错误信息并终止程序。

【样例 1 输入】

```
1 0 1
2 7 2
3 4 5 2 6 3 7 1
4 1 2
5 1 3
6 2 4
7 2 5
8 3 6
9 3 7
```

【样例 1 输出】

```
1 Finished!
2 Case 1: treasure = 13, score ratio = 100.000000%
3 Score ratio: 100.000000%
```

【说明/提示】**【样例 1 解释】**

样例输出对应一种分别清洗宫室 4 与宫室 6 的策略。两次清洗依次取得 6 与 7 的藏宝量，因此最终共取得 13 的藏宝量。

不同的合法策略可能产生不同的样例输出。

【样例 2】

见选手目录下的 `moat/moat2.in`。
该组样例符合测试点 1 的数据范围。

【样例 3】

见选手目录下的 `moat/moat3.in`。
该组样例符合测试点 2 的数据范围。

【样例 4】

见选手目录下的 `moat/moat4.in`。
该组样例符合测试点 3 的数据范围。

【样例 5】

见选手目录下的 `moat/moat5.in`。
该组样例符合测试点 4 的数据范围。

本题为成果评分交互题，附加样例只提供隐藏输入。测试程序的输出由选手程序采用的策略决定。

【数据范围】

对于 100% 的数据，保证 $1 \leq T \leq 4$ ， $2 \leq n \leq 2 \times 10^5$ ， $1 \leq b \leq \min(40, n)$ ， $1 \leq w_u \leq 10^9$ ，每个宫室至多与两个深度比它大 1 的宫室直接相连，王宫的高度 $h \leq 30$ 。
对于同一个测试点，保证 $\sum n \leq 2 \times 10^5$ 。

测试点编号	分值	n	b	特殊性质
1	15	≤ 15	$\leq \min(40, n)$	无
2	20	$\leq 2 \times 10^5$	$= 1$	无
3	20	≤ 30	$\leq \min(40, n)$	A
4	45	$\leq 2 \times 10^5$	$\leq \min(40, n)$	无

特殊性质 A：所有宫内道路构成一条以明光殿为一端的链。

【评分方式】

注意：

- 选手不应当通过非法方式获取交互库的内部信息，如试图直接读取各个宫室的藏宝量，或直接与标准输入、输出流进行交互。此类行为将被视为作弊。
- 交互库不是适应性的。每次调用 `Play` 时，王宫结构与各个宫室最初的藏宝量已经确定。只有选手调用 `Clean` 时，相应宫室的藏宝量才会按照【题目描述】中的规则变为 0。
- 最终的评测交互库与本地测试交互库的实现不同。

对于一组交互合法的测试数据，记所有宫室最初的藏宝量之和为 v ，选手程序最终获取的藏宝量为 s 。该组测试数据的得分比例为

$$p = \min\left(1, \frac{ns}{bv}\right).$$

其中， p 表示该组测试数据的得分比例。若 $p = 1$ ，则该组测试数据取得对应的全部分值。若 $p < 1$ ，则该组测试数据取得对应分值的 p 倍。

若一个测试点包含 T 组测试数据，记第 i 组测试数据的得分比例为 p_i ，则该测试点的得分比例 $\bar{p}$ 为

$$\bar{p} = \frac{1}{T} \sum_{i=1}^T p_i.$$

可以等价地认为，测试点的分值被平均分配给其中的 T 组测试数据，再分别乘以对应的得分比例。若选手程序在任意一组测试数据中发生非法调用，则整个测试点得 0 分。

记表中该测试点的分值为 d ，则取整前的实际得分为 $d\bar{p}$ 。最终得分保留两位小数，并采用向下取整，也就是

$$\frac{\lfloor 100d\bar{p} \rfloor}{100}.$$

云止聆佳响 (echo)

【题目背景】

龙韧于刚，龙吟激荡，云止聆佳响。

【题目描述】

群山的主脉与支脉层层伸展，远望如同一条卧于云间的龙脊。山脊上共有 n 座听音台，其中 1 号听音台位于山口。其余每座听音台都由一条山路与更靠近山口的听音台相连，这些山路使任意两座听音台之间都能互相到达。也就是说，它们构成一棵以 1 号听音台为根的树。

第 u 座听音台封存着一声音高为 a_u 的龙吟。将这座听音台唤醒时，它能够为群山中的声音增添 w_u 的吟响值。

为了让沉寂的龙吟重新贯穿群山，你需要沿着一条从山口向龙脊深处延伸的道路，依次唤醒若干座听音台 $p_1, p_2, \dots, p_s$ 。对于每个 $1 \leq i < s$ ，从山口前往 p_{i+1} 时必须经过 p_i 。换言之， p_i 是 p_{i+1} 的祖先。被唤醒的听音台不必在山路上直接相邻，但必须依次位于同一条道路上。

依次唤醒的听音台共同奏成一段龙吟。为了使每一声音都能清晰传向云间，要求龙吟中任意相邻两座听音台的音高不同。

对于一段龙吟中相邻的两座听音台：

- 若 $a_{p_i} < a_{p_{i+1}}$ ，称此处的龙吟正在上扬。
- 若 $a_{p_i} > a_{p_{i+1}}$ ，称此处的龙吟正在低回。

当连续两处的龙吟一处上扬、另一处低回时，龙吟会在它们之间发生一次激荡。也就是说，对于 $2 \leq i < s$ ，当

$$(a_{p_i} - a_{p_{i-1}})(a_{p_{i+1}} - a_{p_i}) < 0$$

时，龙吟在 p_i 处发生一次激荡。

只唤醒一座听音台也能奏成一段合法的龙吟，此时不会发生激荡。

一段龙吟的吟响值为其中所有听音台贡献的吟响值之和。你需要在发生激荡的次数不超过 k 的所有非空龙吟中，求出吟响值的最大值。

【输入格式】

从文件 `echo.in` 中读入数据。

本题包含多组测试数据。

输入的第一行包含两个整数 c, T ，分别表示测试点编号与测试数据组数。 $c = 0$ 表示该测试点为样例。

接下来依次输入每组测试数据。对于每组测试数据：

第一行包含两个整数 n, k ，分别表示听音台的数量与允许发生激荡的最大次数。

第二行包含 $n - 1$ 个整数 $p_2, p_3, \dots, p_n$ 。对于每个 $2 \leq u \leq n$ ， p_u 表示从听音台 u 前往山口时经过的第一座听音台。

第三行包含 n 个整数 $a_1, a_2, \dots, a_n$ ，表示每座听音台记录的音高。

第四行包含 n 个正整数 $w_1, w_2, \dots, w_n$ ，表示每座听音台贡献的吟响值。

【输出格式】

输出到文件 `echo.out` 中。

对于每组测试数据，输出一行一个整数，表示满足要求的龙吟的最大吟响值。

【样例 1 输入】

```
1 0 2
2 6 1
3 1 1 2 2 3
4 4 2 7 6 1 5
5 3 5 4 6 7 8
6 5 0
7 1 2 3 4
8 3 1 4 2 5
9 2 8 4 7 3
```

【样例 1 输出】

```
1 15
2 18
```

【说明/提示】

【样例 1 解释】

对于第一组测试数据，可以依次唤醒听音台 1, 2, 5。对应的音高依次为 4, 2, 1，龙吟始终低回，没有发生激荡，得到的吟响值为 $3 + 5 + 7 = 15$ 。

也可以依次唤醒听音台 1, 3, 6。对应的音高依次为 4, 7, 5，龙吟发生一次激荡，得到的吟响值为 $3 + 4 + 8 = 15$ 。可以证明不存在吟响值更大的合法龙吟。

对于第二组测试数据，不允许发生激荡。依次唤醒听音台 2, 4, 5 时，音高依次为 1, 2, 5，得到的吟响值为 $8 + 7 + 3 = 18$ ，这是能够得到的最大值。

【样例 2】

见选手目录下的 *echo/echo2.in* 与 *echo/echo2.ans*。
该组样例符合测试点 1 ~ 3 的数据范围。

【样例 3】

见选手目录下的 *echo/echo3.in* 与 *echo/echo3.ans*。
该组样例符合测试点 4 ~ 7 的数据范围。

【样例 4】

见选手目录下的 *echo/echo4.in* 与 *echo/echo4.ans*。
该组样例符合测试点 8 ~ 11 的数据范围。

【样例 5】

见选手目录下的 *echo/echo5.in* 与 *echo/echo5.ans*。
该组样例符合测试点 12 ~ 15 的数据范围。

【样例 6】

见选手目录下的 *echo/echo6.in* 与 *echo/echo6.ans*。
该组样例符合测试点 16 ~ 20 的数据范围。

【数据范围】

对于 100% 的数据, 保证 $1 \leq T \leq 20$, $1 \leq n \leq 2 \times 10^5$, $0 \leq k \leq 8$, $1 \leq a_u, w_u \leq 10^9$ 。
对于同一个测试点, 保证 $\sum n \leq 2 \times 10^5$ 。

测试点编号	n	k	特殊性质
1 ~ 3	≤ 20	≤ 8	无
4 ~ 7	$\leq 2 \times 10^3$		
8 ~ 11	$\leq 2 \times 10^5$	$= 0$	B
12 ~ 15		≤ 8	A
16 ~ 20			无

特殊性质 A: 所有听音台构成一条以 1 号听音台为一端的链。

特殊性质 B: $k = 0$ 。